# Reinforcement Learning

## Policy Gradients

SJTU

# Outline


1. The RL Objective and Policy Gradient
2. REINFORCE Algorithm
3. Understanding Policy Gradients
4. Variance Reduction
5. Off-Policy Policy Gradients
6. Practical Implementation
7. Advanced Policy Gradients (Preview)
8. Summary

# Recap: The RL Objective

- Policy $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ parameterized by $\theta$ (e.g., neural network weights)
- Trajectory: $\tau = (\mathbf{s}_1, \mathbf{a}_1, \mathbf{s}_2, \mathbf{a}_2, \ldots, \mathbf{s}_T, \mathbf{a}_T)$
- Trajectory distribution:

$$p_\theta(\tau) = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)\, p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

- **RL objective** — maximize expected total reward:

$$\theta^* = \arg\max_\theta J(\theta) = \arg\max_\theta \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[ \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

# How to Optimize $J(\theta)$?

- $J(\theta)$ involves an **expectation** over trajectories
- Trajectories depend on $\theta$ through $\pi_\theta$
- Cannot simply differentiate through the environment dynamics
- **Key idea:** use the log-derivative trick to compute $\nabla_\theta J(\theta)$

Policy Gradient: directly differentiate the RL objective

# The Log-Derivative Trick

## Identity

For any distribution $p_\theta(x)$:

$$p_\theta(x)\,\nabla_\theta \log p_\theta(x) = p_\theta(x) \cdot \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} = \nabla_\theta p_\theta(x)$$

This is useful because:

- We can **sample** from $p_\theta$ (run the policy)
- We can **evaluate** $\nabla_\theta \log p_\theta$ (backprop through the policy network)
- We **cannot** easily compute $\nabla_\theta p_\theta$ directly (involves the dynamics)

# Deriving the Policy Gradient

Starting from $J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[r(\tau)]$ where $r(\tau) = \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$:

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau)\, r(\tau)\, d\tau \\
&= \int \nabla_\theta p_\theta(\tau)\, r(\tau)\, d\tau \\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, r(\tau)\, d\tau \\
&= \mathbb{E}_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau)\, r(\tau)]
\end{aligned}
$$

# Simplifying $\nabla_\theta \log p_\theta(\tau)$

- Expand the trajectory distribution:

$$\log p_\theta(\tau) = \log p(\mathbf{s}_1) + \sum_{t=1}^{T} \Big[ \log \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) + \log p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) \Big]$$

- Taking $\nabla_\theta$, the dynamics and initial state terms **vanish**:

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$$

We don't need to know the dynamics $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$!

# The Policy Gradient Theorem

## Policy Gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[ \left( \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) \right) \left( \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

- **Model-free:** no knowledge of dynamics required
- Only need to evaluate $\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s})$ — the **score function**
- Can approximate the expectation with **samples**:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left( \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left( \sum_{t=1}^{T} r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

# The REINFORCE Algorithm

## REINFORCE (Williams, 1992)

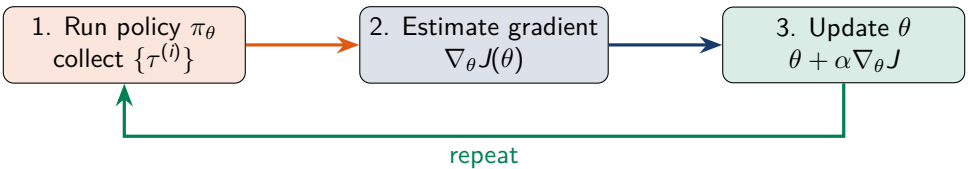
1. **Sample** $N$ trajectories $\{\tau^{(i)}\}_{i=1}^{N}$ by running $\pi_\theta$
2. **Estimate** the policy gradient:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left( \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left( \sum_{t=1}^{T} r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

3. **Update**: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

- Repeat until convergence
- This is a **Monte Carlo** method — uses complete trajectory returns

# REINFORCE: Pictorial Overview



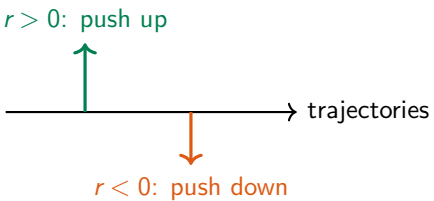

- On-policy: must collect **new** samples after each update
- Each gradient step uses samples from the **current** policy $\pi_\theta$
- After updating $\theta$, old samples are no longer valid

# What Does the Policy Gradient Do?

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left( \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) r(\tau^{(i)})$$

**Intuition:**

- Good trajectory ($r(\tau) > 0$): **increase** probability of those actions
- Bad trajectory ($r(\tau) < 0$): **decrease** probability of those actions
- It's **trial and error**: try things, do more of what works



# Comparison to Maximum Likelihood

## Maximum Likelihood

$$\nabla_\theta J_{\mathrm{ML}} = \frac{1}{N} \sum_{i=1}^{N} \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})$$

- Make **all** demonstrated actions more likely
- Data comes from a fixed dataset

## Policy Gradient

$$\nabla_\theta J = \frac{1}{N} \sum_{i=1}^{N} \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \, \hat{Q}_t^{(i)}$$

- Make **good** actions more likely, **bad** actions less likely
- Data comes from running the policy

Policy gradient = **reward-weighted** maximum likelihood

# Example: Gaussian Policy

For continuous actions, a common choice:

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \mathcal{N}(f_{\text{nn}}(\mathbf{s};\, \theta),\ \boldsymbol{\Sigma})$$

- $f_{\text{nn}}$: neural network outputs the **mean** action
- $\boldsymbol{\Sigma}$: covariance matrix (can be fixed or learned)
- Log-probability:

$$\log \pi_\theta(\mathbf{a}|\mathbf{s}) = -\frac{1}{2} \left(\mathbf{a} - f_{\text{nn}}(\mathbf{s})\right)^\top \boldsymbol{\Sigma}^{-1} \left(\mathbf{a} - f_{\text{nn}}(\mathbf{s})\right) + \text{const}$$

- $\nabla_\theta \log \pi_\theta$ is computed via **backpropagation** through $f_{\text{nn}}$
- For discrete actions: use softmax output + cross-entropy

# Policy Gradients and Partial Observability

- Recall: in POMDPs, we have observations $\mathbf{o}_t$ instead of states $\mathbf{s}_t$
- **Good news:** policy gradient works without modification!
- Simply replace $\mathbf{s}_t$ with $\mathbf{o}_t$ in the policy: $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

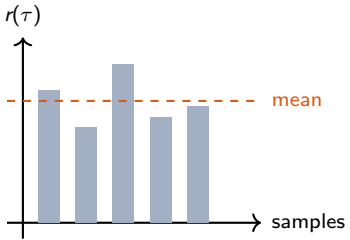
## Why This Works

- The derivation never required the Markov property
- We only needed: (1) sample trajectories, (2) evaluate $\log \pi_\theta$
- Both work with observations just as well as with states

*Note: the policy may be suboptimal without full state, but the gradient is still valid.*

# The High Variance Problem

- The policy gradient estimator has **very high variance**
- Consider: all rewards are positive
  - All trajectories push actions **up**
  - Good trajectories push **more**, bad push **less**
  - Relative differences are small compared to absolute values
- With finite samples, the gradient estimate is **very noisy**
- Needs many samples to get a useful signal




All positive → all push up!

# Causality: Reward-to-Go

- Current formulation: each action is weighted by the **total** trajectory reward
- But: action at time $t$ **cannot affect** rewards at times $t' < t$!
- **Causality trick:** only use future rewards ("reward-to-go"):

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \, \hat{Q}_t^{(i)}$$

where:

$$\hat{Q}_t^{(i)} = \sum_{t'=t}^{T} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \qquad \text{(reward-to-go)}$$

- This reduces variance **without introducing bias**
- Each action is credited only for rewards it can actually influence

# Baselines

- Subtract a **baseline** $b$ from the reward to center the gradient signal:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t} \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left( \hat{Q}_t^{(i)} - b \right)$$

- Common choice: $b = \frac{1}{N} \sum_{i=1}^{N} r(\tau^{(i)})$ (average return)
- Actions better than average $\rightarrow$ increase probability
- Actions worse than average $\rightarrow$ decrease probability

**Critical question:** Does subtracting $b$ introduce bias?

# Baselines Are Unbiased

## Proof

We need to show: $\mathbb{E}[\nabla_\theta \log p_\theta(\tau) \cdot b] = 0$

$$\begin{aligned}
\mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot b] &= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau) \cdot b \, d\tau \\
&= b \int \nabla_\theta p_\theta(\tau)\, d\tau \\
&= b\, \nabla_\theta \underbrace{\int p_\theta(\tau)\, d\tau}_{=1} \\
&= b \cdot 0 = 0 \quad \checkmark
\end{aligned}$$

**Subtracting any constant baseline does not change the expected gradient, but reduces variance!**

# Optimal Baseline

- Not all baselines reduce variance equally
- The **variance-minimizing** baseline can be derived analytically:

$$b^* = \frac{\mathbb{E}\left[\left(\nabla_\theta \log p_\theta(\tau)\right)^2 \, r(\tau)\right]}{\mathbb{E}\left[\left(\nabla_\theta \log p_\theta(\tau)\right)^2\right]}$$

- This is a **reward weighted by gradient magnitudes**
- In practice, using $b = \frac{1}{N}\sum_i r(\tau^{(i)})$ works well
- Even better: use a **learned value function** $V_\phi(\mathbf{s}_t) \approx \mathbb{E}[\hat{Q}_t|\mathbf{s}_t]$ (actor-critic methods — next lecture)

# Putting It Together: Better Policy Gradient

**Improved REINFORCE with Baseline and Causality**

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left( \hat{Q}_t^{(i)} - b \right)$$

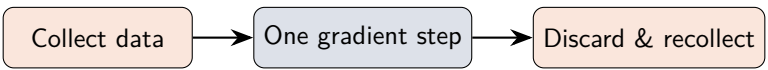
where $\hat{Q}_t^{(i)} = \sum_{t'=t}^{T} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)})$ and $b$ is a baseline (e.g., average return).

Two key variance reduction techniques:

1. **Causality / reward-to-go**: only credit future rewards
2. **Baselines**: center the reward signal around a reference

# The On-Policy Problem

- REINFORCE is **on-policy**: samples must come from $\pi_\theta$
- After each gradient step, $\theta$ changes $\rightarrow$ old samples are invalid
- Neural networks change only a **little** with each gradient step
- Throwing away data after each step is extremely **wasteful**!




*Very inefficient!*

# Importance Sampling

## Key Identity

For any distributions $p(x)$ and $q(x)$:

$$\mathbb{E}_{x\sim p(x)}[f(x)] = \int p(x)f(x)\,dx = \int q(x)\frac{p(x)}{q(x)}f(x)\,dx = \mathbb{E}_{x\sim q(x)}\left[\frac{p(x)}{q(x)}f(x)\right]$$

- We can evaluate an expectation under $p$ using samples from $q$
- The ratio $\frac{p(x)}{q(x)}$ is the **importance weight**
- Useful when we have samples from an **old policy** $\bar{\pi}$ but want to evaluate under $\pi_\theta$

# Importance Sampling for Policy Gradients

- We have samples from $\bar{p}(\tau)$ (old policy), want to estimate $J(\theta)$ under $p_\theta(\tau)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \bar{p}(\tau)} \left[ \frac{p_\theta(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$

- The importance weight simplifies:

$$\frac{p_\theta(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(\mathbf{s}_1)} \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)}}{\cancel{p(\mathbf{s}_1)} \prod_{t=1}^{T} \bar{\pi}(\mathbf{a}_t|\mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)}} = \prod_{t=1}^{T} \frac{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}{\bar{\pi}(\mathbf{a}_t|\mathbf{s}_t)}$$

- Dynamics cancel out! Only need policy ratios.

# Deriving the Policy Gradient with IS

Can we estimate the value of new parameters $\theta'$ using old samples from $\pi_\theta$?

$$J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[ \frac{p_{\theta'}(\tau)}{p_\theta(\tau)} \, r(\tau) \right]$$

Taking the gradient with respect to $\theta'$:

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[ \frac{\nabla_{\theta'} p_{\theta'}(\tau)}{p_\theta(\tau)} \, r(\tau) \right] = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[ \frac{p_{\theta'}(\tau)}{p_\theta(\tau)} \, \nabla_{\theta'} \log p_{\theta'}(\tau) \, r(\tau) \right]$$

Evaluating locally at $\theta' = \theta$: the ratio becomes 1, recovering the original PG:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [\nabla_\theta \log p_\theta(\tau) \, r(\tau)]$$

# The Off-Policy Policy Gradient

When $\theta' \neq \theta$ (using old data from $\pi_\theta$ to update $\pi_{\theta'}$):

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta} \left[ \left( \prod_{t=1}^{T} \frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \right) \left( \sum_t \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \right) \left( \sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

With causality, only **past** importance weights matter:

$$= \mathbb{E}_{\tau \sim p_\theta} \left[ \sum_{t=1}^{T} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \left( \prod_{t'=1}^{t} \frac{\pi_{\theta'}(\mathbf{a}_{t'}|\mathbf{s}_{t'})}{\pi_\theta(\mathbf{a}_{t'}|\mathbf{s}_{t'})} \right) \hat{Q}_t \right]$$

- Problem: product of ratios grows **exponentially** in $T$!

# First-Order Approximation

- The product of IS weights $\prod_{t'=1}^{t} \frac{\pi_{\theta'}}{\pi_\theta}$ causes very high variance
- **Approximation:** ignore the state visitation change, keep only action ratio:

**On-policy PG:**

$$\frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta \cdot \hat{Q}_t$$

**Off-policy PG (approx):**

$$\frac{1}{N} \sum_i \sum_t \frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \nabla_{\theta'} \log \pi_{\theta'} \cdot \hat{Q}_t$$

- Only a **single** ratio per time step (not a product)
- This approximation ignores the change in state distribution
- Leads to **policy iteration** methods (covered later in the course)

# Implementing PG with Automatic Differentiation

- Computing $\nabla_\theta \log \pi_\theta$ explicitly is inefficient
- Instead, define a **pseudo-loss** whose gradient equals the policy gradient:

$$\tilde{J}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \, \hat{Q}_t^{(i)}$$

- $\nabla_\theta \tilde{J}(\theta)$ gives the policy gradient (since $\hat{Q}_t$ is treated as a constant)
- This is just **weighted maximum likelihood**!
- Use any autodiff framework (PyTorch, JAX) to compute the gradient

Pseudo-loss = reward-weighted log-likelihood

# Implementation: Discrete Actions

**Maximum likelihood** (supervised learning):

```
# actions: (N*T,) tensor of action indices
# states:  (N*T, Ds) tensor of states
logits = policy(states)
loss = cross_entropy(logits, actions)
loss.backward()
```

**Policy gradient** — just add weights:

```
# q_values: (N*T,) tensor of reward-to-go
logits = policy(states)
log_probs = -cross_entropy(logits, actions)
loss = -(log_probs * q_values).mean()
loss.backward()
```

# Practical Tips for Policy Gradients

**Challenges:**

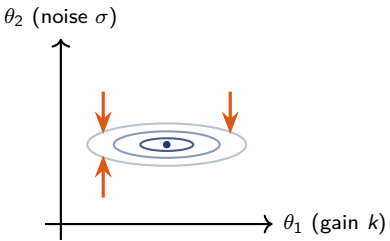
- Gradient has **high variance**
  - Not the same as supervised learning
  - Gradients will be very noisy
- Sensitive to **learning rate**
- On-policy = **sample inefficient**

**Best Practices:**

- Use **large batch sizes** (many trajectories)
- Use **baselines** (always!)
- Use **reward-to-go** (causality)
- Try adaptive optimizers (Adam)
- Normalize rewards / advantages

# What Else Can Go Wrong?

- Even with variance reduction, vanilla PG can be **unstable**
- **Problem:** gradient descent in parameter space $\neq$ gradient descent in policy space
- A small change in $\theta$ may cause a **large** change in $\pi_\theta$
- Different parameters affect the policy **differently**
- Example: changing variance parameter $\sigma$ has much larger effect than shifting mean $\mu$




Ill-conditioned landscape

# Natural / Covariant Policy Gradient

- Standard gradient: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ (step in **parameter** space)
- Better: constrain step size in **policy** space using KL-divergence:

$$\theta' \leftarrow \arg\max_{\theta'} (\theta' - \theta)^\top \nabla_\theta J(\theta) \quad \text{s.t.} \quad D_{\mathrm{KL}}(\pi_{\theta'} \| \pi_\theta) \leq \epsilon$$

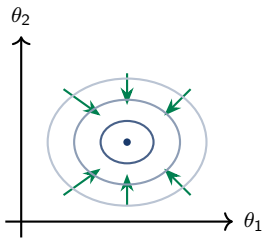
- Solution: the **natural gradient**:

$$\theta \leftarrow \theta + \alpha \mathbf{F}^{-1} \nabla_\theta J(\theta)$$

- $\mathbf{F} = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s})\, \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s})^\top\right]$ is the **Fisher information matrix**

# Trust Region Methods

- **Natural gradient**: pick learning rate $\alpha$
- **Trust region policy optimization (TRPO)**: pick constraint $\epsilon$
  - Can solve for optimal $\alpha$ given $\epsilon$
  - Conjugate gradient for $\mathbf{F}^{-1}\nabla_\theta J$
- **PPO**: approximates the trust region with clipping
  - Simpler to implement
  - Widely used in practice



# Summary

- **Policy gradient:** directly optimize the RL objective via gradient ascent
  - Log-derivative trick eliminates need for dynamics model
  - REINFORCE: sample, estimate gradient, update (on-policy)
- **Understanding PG:** reward-weighted maximum likelihood
  - Works with partial observability
  - Gaussian policies for continuous actions
- **Variance reduction** is critical:
  - Causality / reward-to-go: only credit future rewards
  - Baselines: subtract average return (unbiased!)
- **Off-policy PG:** importance sampling enables data reuse
  - But full IS has exponential variance; first-order approximation helps
- **Advanced methods:** natural gradient, TRPO, PPO for stability

**Next lecture:** Actor-Critic methods — using value functions to further reduce variance

# Suggested Readings

- **Classic papers:**
  - Williams (1992). Simple statistical gradient-following algorithms for connectionist RL: introduces REINFORCE
  - Baxter & Bartlett (2001). Infinite-horizon policy-gradient estimation
  - Peters & Schaal (2008). RL of motor skills with policy gradients: optimal baselines and natural gradient
- **Deep RL policy gradient papers:**
  - Levine & Koltun (2013). Guided policy search: deep RL with importance sampled PG
  - Schulman et al. (2015). Trust region policy optimization (TRPO)
  - Schulman et al. (2017). Proximal policy optimization (PPO)

# Thank You

Questions?